

CNN Foundations

- Convolution,
- Spatial Arithmetic,
- Receptive Field

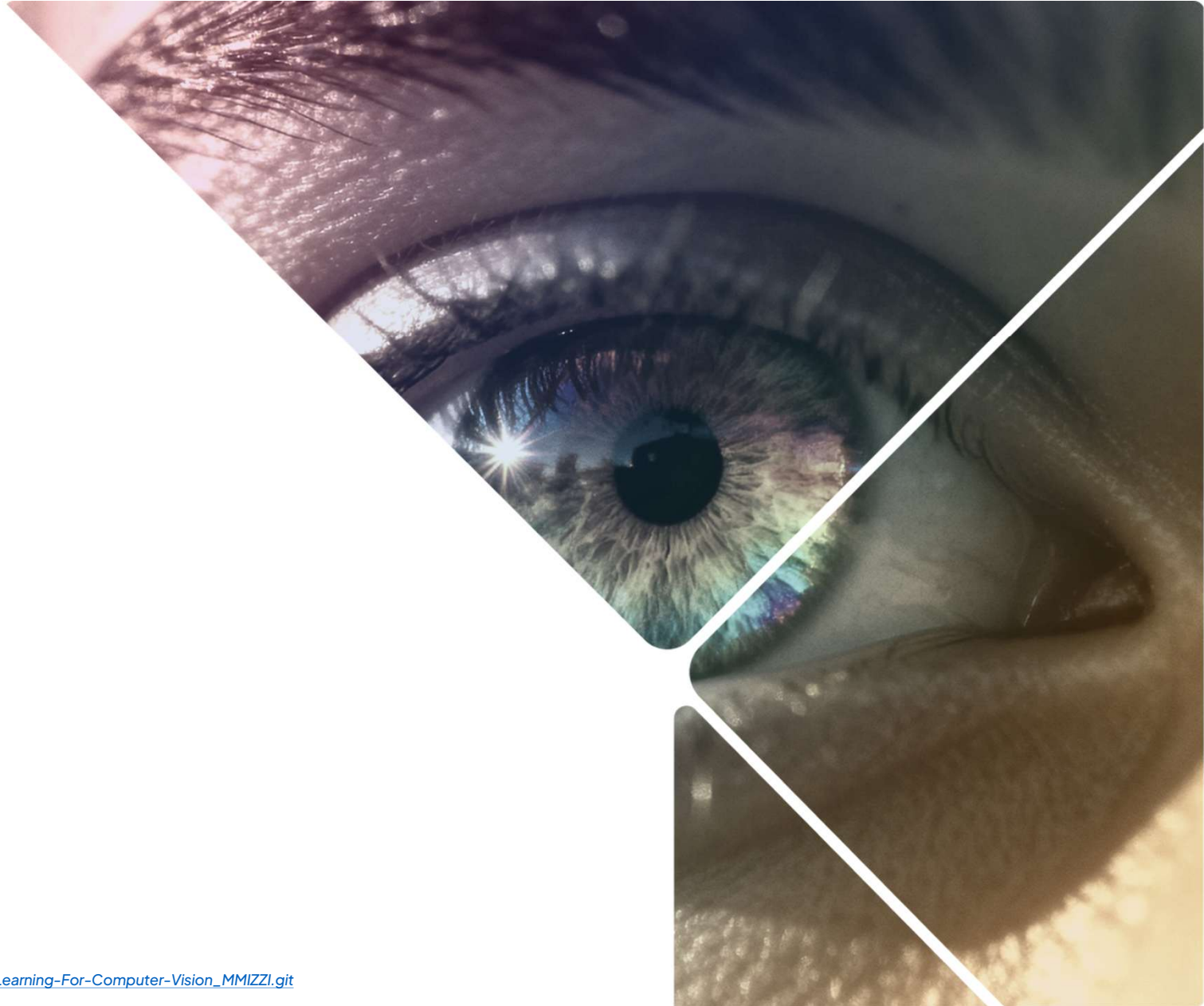
Study Pack:

- Notes
- Quiz
- A worked Example (Jupyter Notebook)
- Simulator (Gradio platform)

Michael Mizzi

May 2026

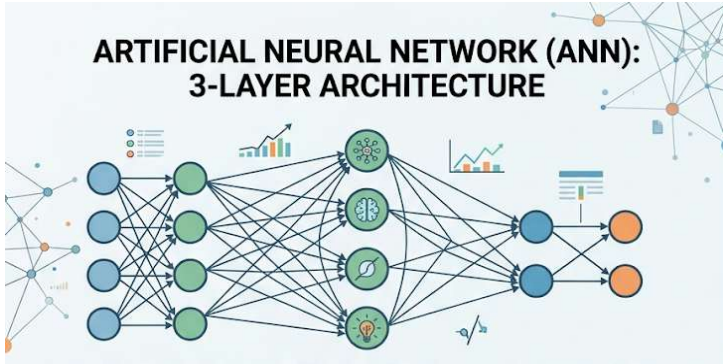
Repository Link: https://github.com/micmizzi/ARI5118-Deep-Learning-For-Computer-Vision_MMIZZI.git



Why Traditional ANNs Fail?

Struggling with visual data

ARTIFICIAL NEURAL NETWORK (ANN): 3-LAYER ARCHITECTURE



Example:

3-layer: $4i \rightarrow 4H \rightarrow 2o$

→ 28x28 pixel: 3,150 par.

→ 128x128 pixel: 65,550 par.

4-layer: 784i / 128H / 64H / 10o

→ 28x28 pixel: ~110k par.



01

Parameter Explosion

Small images need millions of weights, creating massive memory usage and overfitting risk.

02

Spatial Topology

Networks treat images as flat vectors, ignoring that nearby pixels are highly correlated.

03

Lack of Invariance

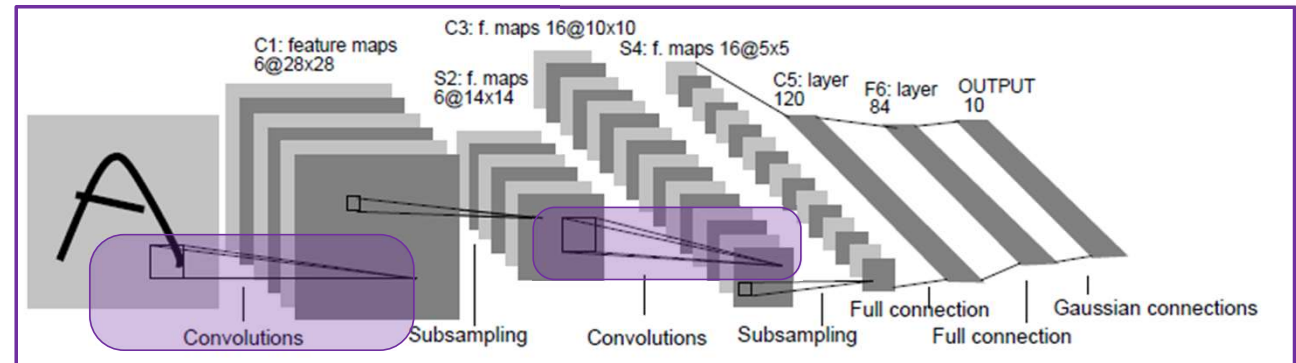
Networks lack translation invariance, failing to recognise objects shifted by a few pixels.

CNN: The core building blocks & Spatial Arithmetic

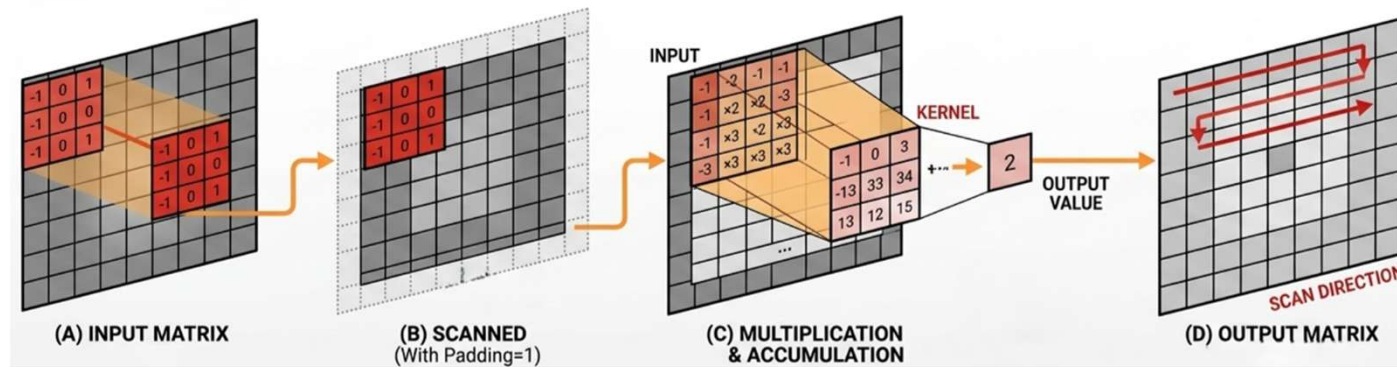
- From an 2D image to a 1D vector
- From input (*i*) to output (*o*)
 - Convolutions
 - Hyperparameters:
 - Kernel,
 - padding,
 - stride,
 - dilation
- The Effective Receptive Field

Architecture of LeNet-5

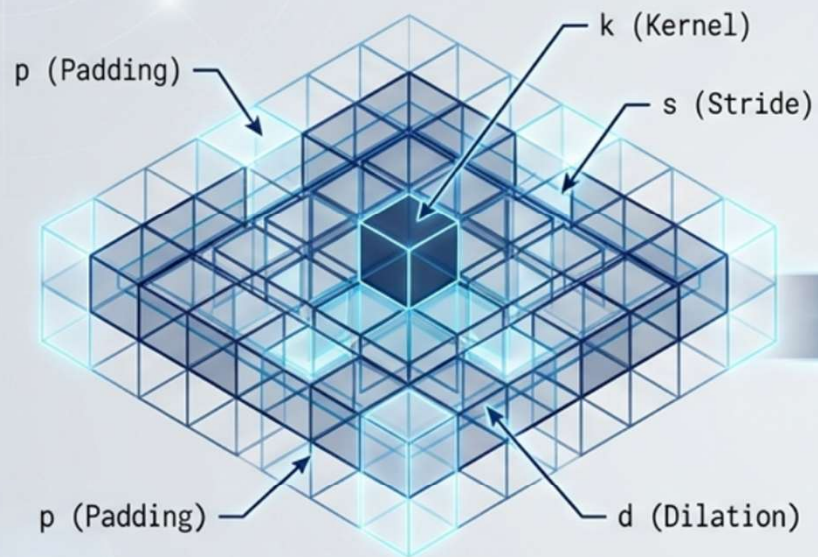
Gradient-Based Learning Applied to Document Recognition
Yann Lecun, L'eon Bottou, Yoshua Bengio, And Patrick Haffner
1998



~60,000 parameters

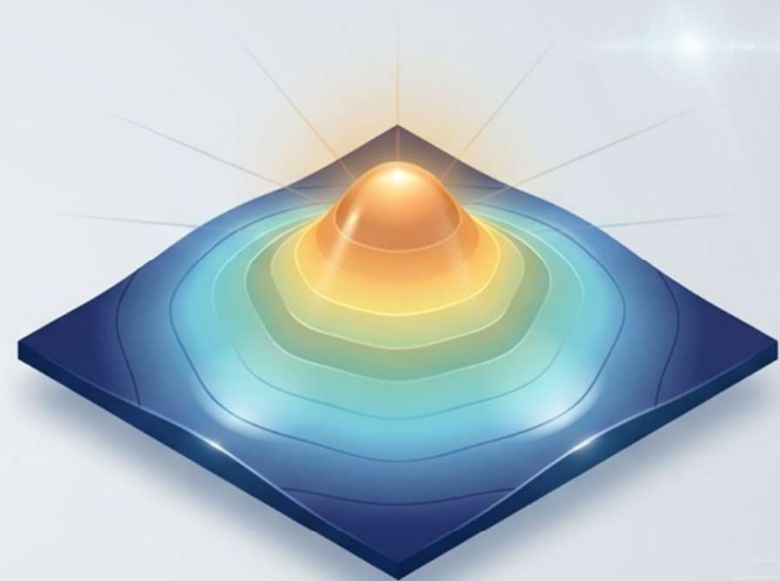


The Micro: Spatial Arithmetic



Master the pixel-level spatial operations that manipulate input matrices.

The Macro: Systemic Perception



Understand how these micro-operations compound to form the network's Effective Receptive Field.

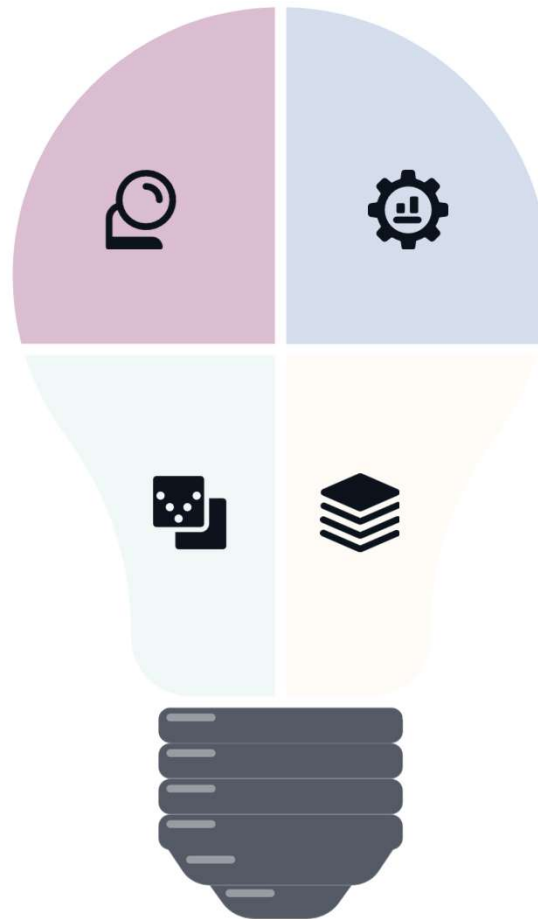
CNNs Mirror Biological Vision

Local Vision

CNNs improve upon standard ANNs by focusing on small, local regions

Consistency

The same kernel weight across the entire image. Detect a specific feature consistently, regardless of its exact location.



Efficiency

Neuron connects only to a small grid which drastically reduces total parameters.

Hierarchy

Structural design enables a sophisticated, efficient extraction pipeline. Transforms raw pixel data into complex, high-level visual intelligence.



.....a side note

Input Matrix I:

```
[[6 3 7 4 6]
 [9 2 6 7 4]
 [3 7 7 2 5]
 [4 1 7 5 1]
 [4 0 9 5 8]]
```

Kernel K:

```
[[-2 0]
 [ 0 -1]]
```

Padded Input Matrix:

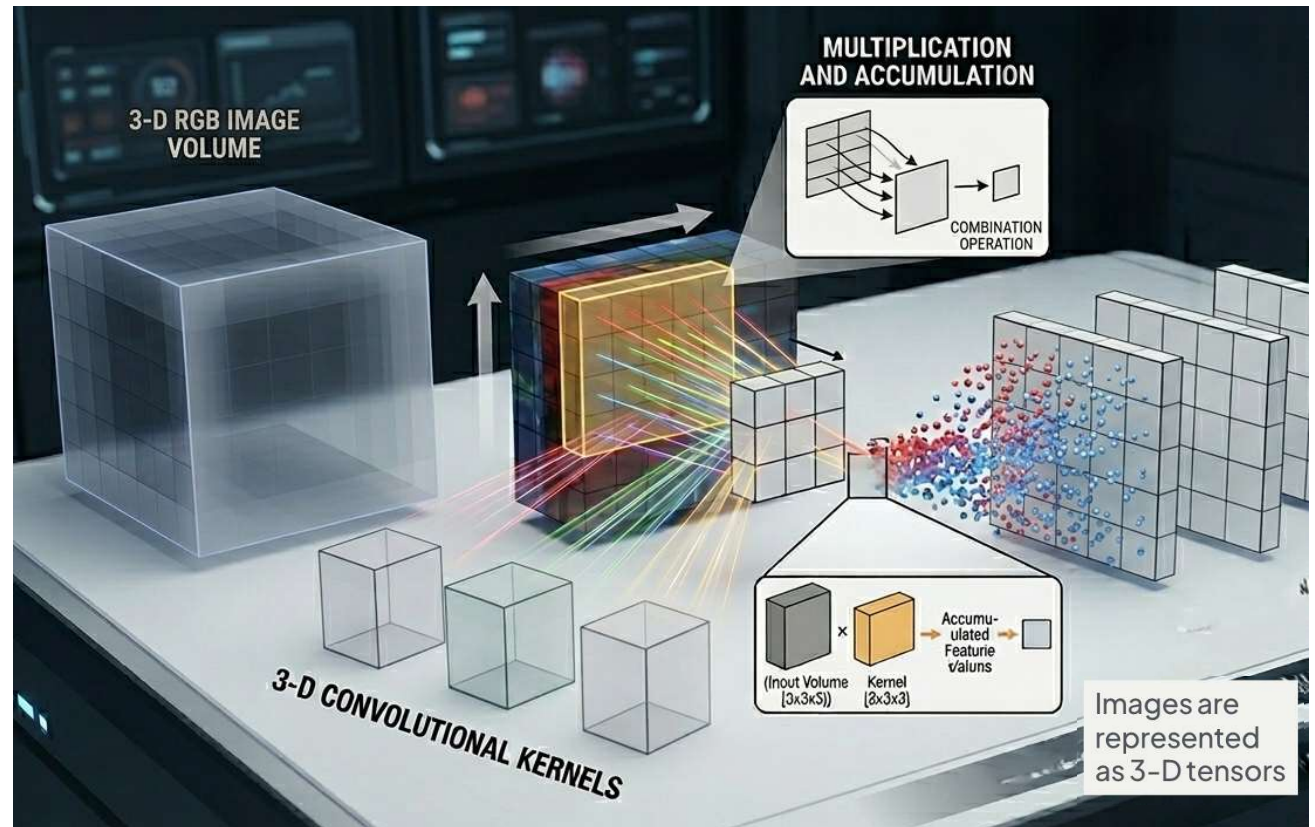
```
[[0 0 0 0 0 0 0]
 [0 6 3 7 4 6 0]
 [0 9 2 6 7 4 0]
 [0 3 7 7 2 5 0]
 [0 4 1 7 5 1 0]
 [0 4 0 9 5 8 0]
 [0 0 0 0 0 0 0]]
```

Effective Dilated Kernel:

```
[[-2. 0. 0.]
 [ 0. 0. 0.]
 [ 0. 0. -1.]]
```

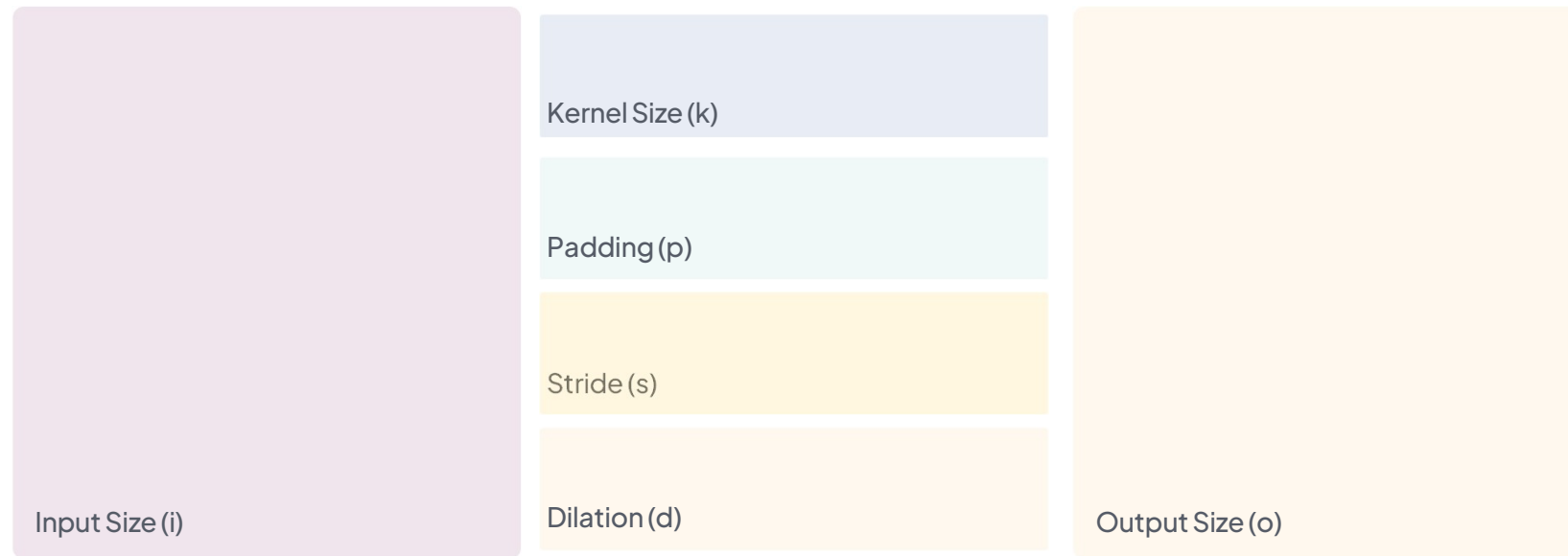
Final Output Matrix after Convolution:

```
[[-2. -6. -7. -4. 0.]
 [-7. -19. -8. -19. -8.]
 [-1. -25. -9. -13. -14.]
 [ 0. -15. -19. -22. -4.]
 [ 0. -8. -2. -14. -10.]]
```



Reference to the Jupyter notebook “[walkthrough.ipynb](#)”

Mastering Spatial Dimension Arithmetic



Output dimensions rely on the exact mathematical relationships between these variables. These govern information flow and computational efficiency within the network architecture.

Convolution Operation Mechanics

$$I = \begin{bmatrix} 6 & 3 & 7 & 4 & 6 \\ 9 & 2 & 6 & 7 & 4 \\ 3 & 7 & 7 & 2 & 5 \\ 4 & 1 & 7 & 5 & 1 \\ 4 & 0 & 9 & 5 & 8 \end{bmatrix}$$

$$K = \begin{bmatrix} -2 & 0 \\ 0 & -1 \end{bmatrix}$$

Sub-matrix:
Step 1

$$\begin{bmatrix} 6 & 3 \\ 9 & 2 \end{bmatrix}$$

1

Sliding Window

kernel moves across the input feature map systematically.

2

Multiplication

At each position perform element-wise multiplication

3

Summation

Sum of products to produce a single numerical value.

4

Integration

Distinct kernels of each colour channel and then summed

$$\text{Output} = (6 \times -2) + (3 \times 0) + (9 \times 0) + (2 \times -1)$$

$$\text{Output} = -12 + 0 + 0 - 2$$

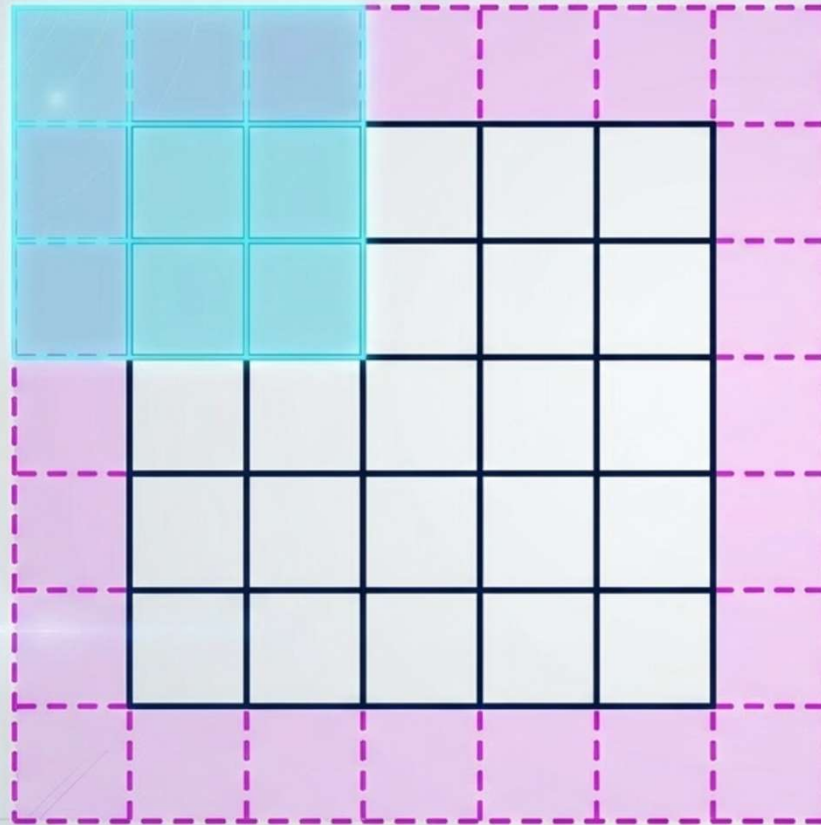
$$\text{Output} = -14$$

Final Output:

$$\begin{bmatrix} -14 & -12 & -21 & -12 \\ -25 & -11 & -14 & -19 \\ -7 & -21 & -19 & -5 \\ -8 & -11 & -19 & -18 \end{bmatrix}$$

Padding

The Phantom Border: Preserving Resolution with Padding (p)



The Edge Penalty

In standard convolution, corner pixels are only sampled once, while central pixels are sampled multiple times. The image shrinks and edge data is lost.

The Solution

Padding the input with zeros artificially expands the effective input size to $i + 2p$.

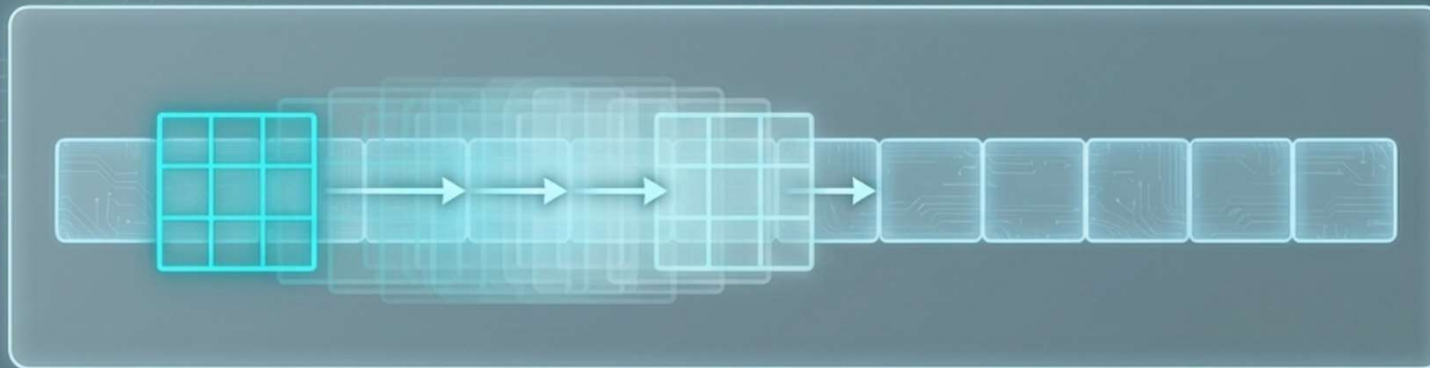
Same Padding

By setting $p = \text{floor}(k/2)$, the spatial output perfectly matches the input resolution.

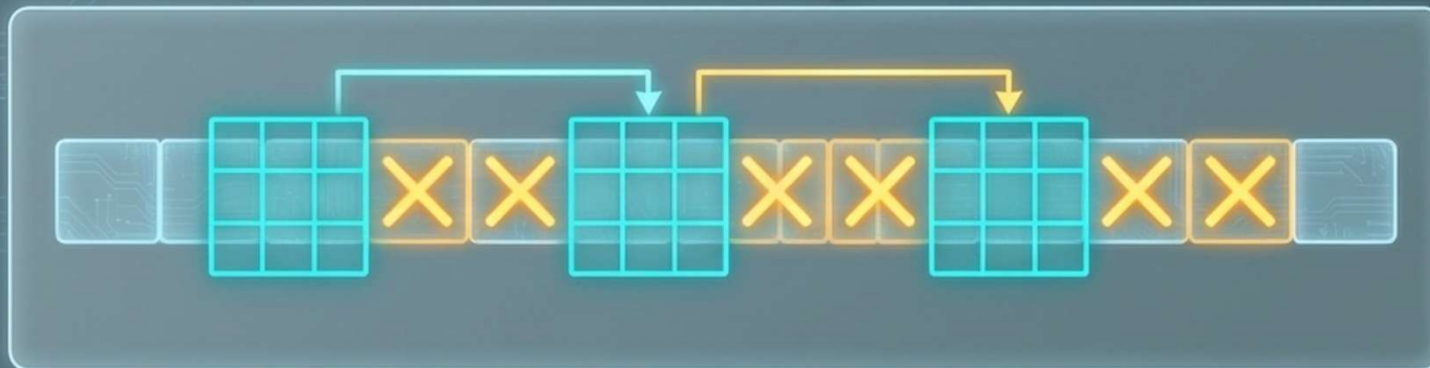
Stride

Adjusting the Focal Step: Subsampling with Stride (s)

Unit Strides ($s = 1$)



Strided Subsampling ($s = 2$)



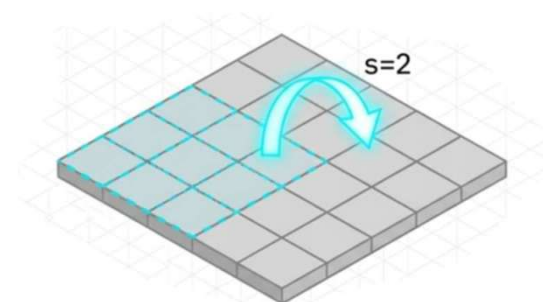
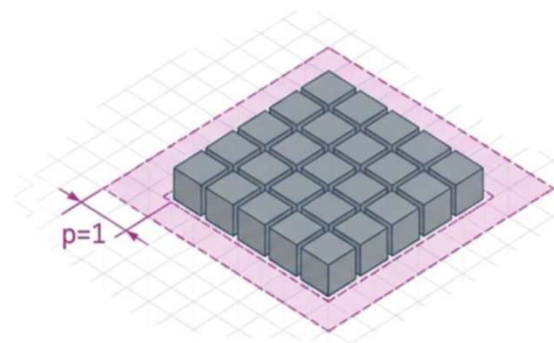
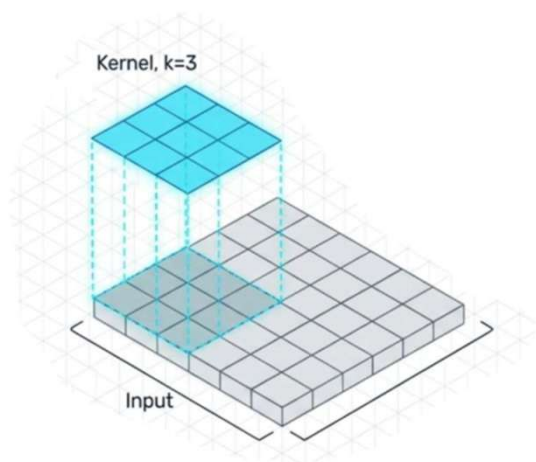
Efficiency

Strides ($s > 1$) constitute a form of spatial subsampling. Instead of translating by increments of 1, the kernel hops.

The Math

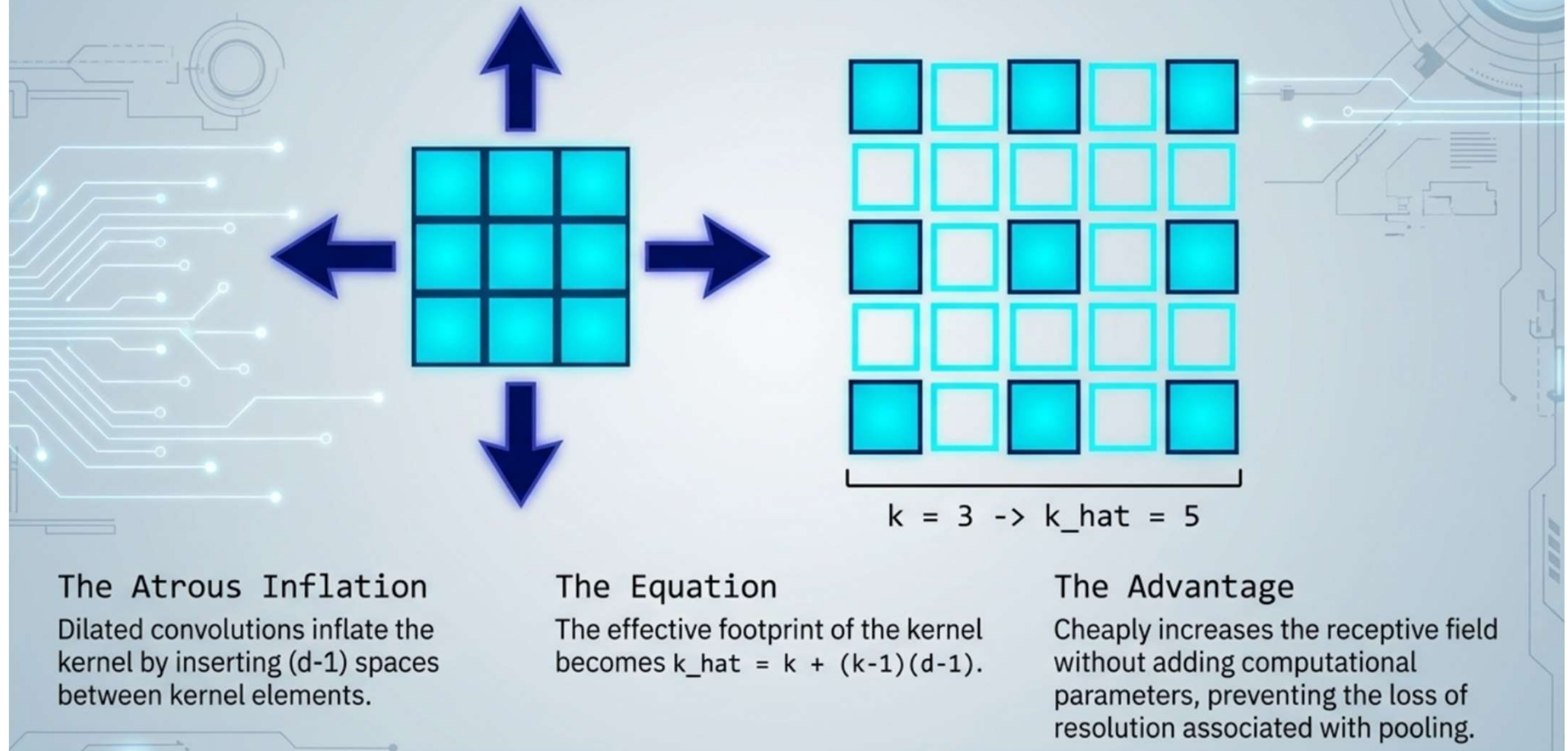
By dividing the spatial plane by s , the output dimensions shrink exponentially, reducing memory footprint and compute load deeper in the network.

$$o = \text{floor}((i + 2p - k) / s) + 1$$

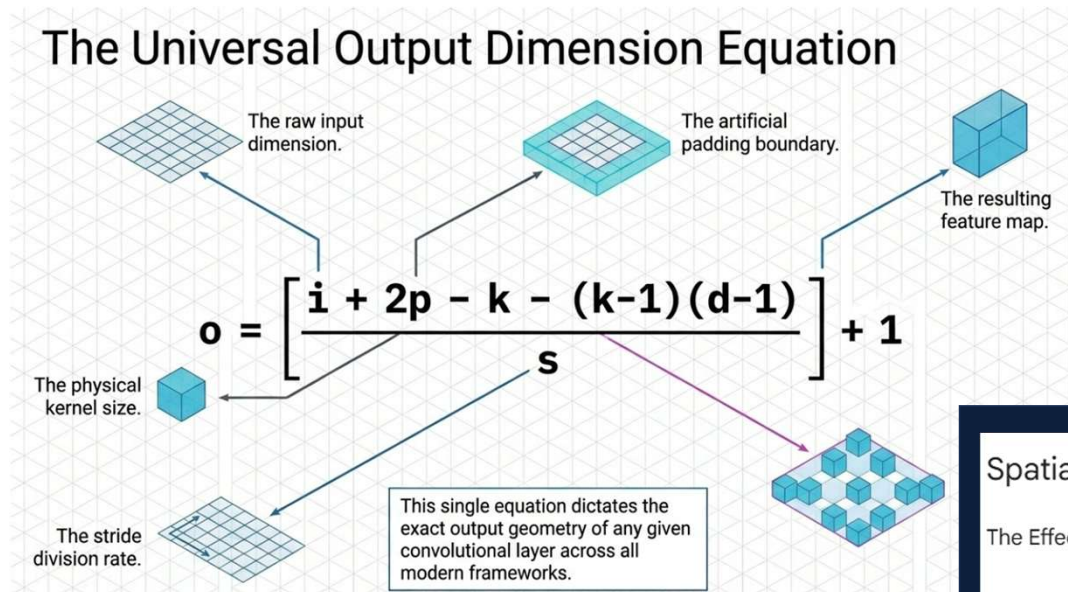


Dilation

The Net Expansion: Multi-Scale Context via Dilation (d)



Compute Output Dimensions for Layers



Input (I): 5×5 matrix.

Kernel (K): 2×2 matrix.

Stride (S): 1.

Padding (P): 1.

Dilation (D): 2 (increases the effective footprint of the kernel).

Spatial Arithmetic

The Effective Kernel Size (K_{eff}) due to dilation ($D = 2$) is represented as follows:

$$K_{eff} = K + (K - 1)(D - 1) = 2 + (2 - 1)(2 - 1) = 3$$

The kernel behaves like a 3×3 filter where only the corners contain the weights:

$$\text{Effective Kernel} = \begin{bmatrix} -2 & \cdot & 0 \\ \cdot & \cdot & \cdot \\ 0 & \cdot & -1 \end{bmatrix}$$

Based on the above kernel and dilation adjustments the Output Dimensions (O) is equal to:

$$O = \frac{I + 2P - K_{eff}}{S} + 1 = \frac{5 + 2(1) - 3}{1} + 1 = 5$$

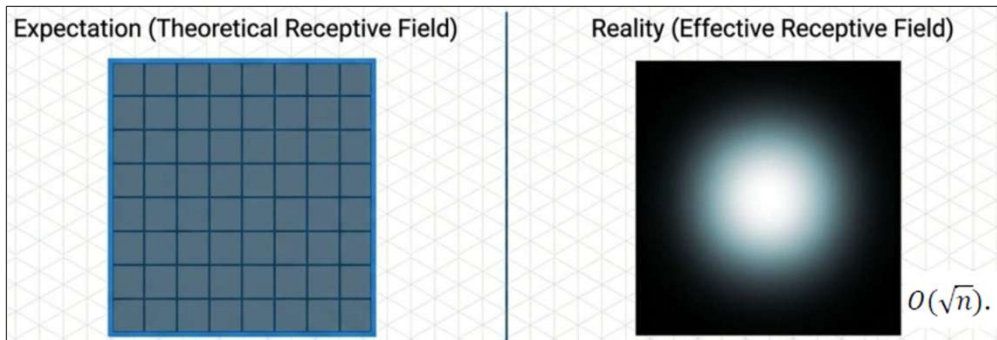
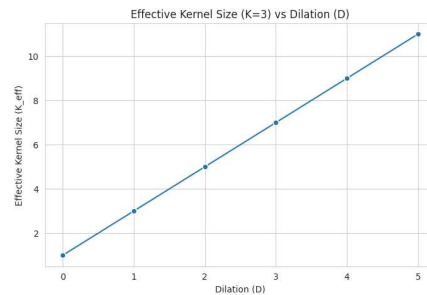
Thus the output is a 5×5 matrix.

Step-by-step guide to calculate output size → [“walkthrough.ipynb”](#)

Expand the Receptive Field with Stride /Dilation

Increase spatial reach while keeping kernel parameters stable:

$$r_0 = \sum_{l=1}^L \left((k_l - 1) \prod_{i=1}^{l-1} s_i \right) + 1$$



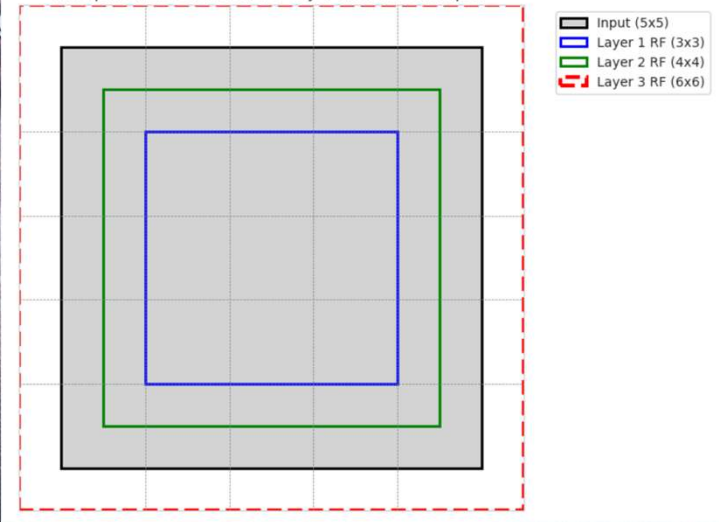
Original Input Size: 5x5
Original Kernel Size: 2x2

--- Layer 1 (using setup parameters S=1, P=1, D=2) ---
Parameters: Stride (S1)=1, Padding (P1)=1, Dilation (D1)=2
Effective Kernel Size (K_eff1): 3x3
Output Size (O1): 5x5
Receptive Field of one neuron in Layer 1: 3x3 on original input.

--- Layer 2 (assuming S=2, P=0, D=1) ---
Input Size to Layer 2 (Output of Layer 1): 5x5
Parameters: Stride (S2)=2, Padding (P2)=0, Dilation (D2)=1
Effective Kernel Size (K_eff2): 2x2
Output Size (O2): 2x2
Receptive Field of one neuron in Layer 2: 4x4 on original input.

--- Layer 3 (assuming S=1, P=0, D=1) ---
Input Size to Layer 3 (Output of Layer 2): 2x2
Parameters: Stride (S3)=1, Padding (P3)=0, Dilation (D3)=1
Effective Kernel Size (K_eff3): 2x2
Output Size (O3): 1x1
Receptive Field of one neuron in Layer 3: 6x6 on original input.

Receptive Field Growth in a 3-Layer CNN (on 5x5 Input)



Demo

Kernel → ‘Blur effect’

Blur (Box)

Kernel Size (N*N)

3

5

[[0.111 0.111 0.111]
[0.111 0.111 0.111]
[0.111 0.111 0.111]]

PADDING

1

5

STRIDE

2

5

DILATION

2

5

Output 2

Kernel type: Blur (Box)

Setup 2 Details

CONVOLUTION SETUP 2
DETAILS

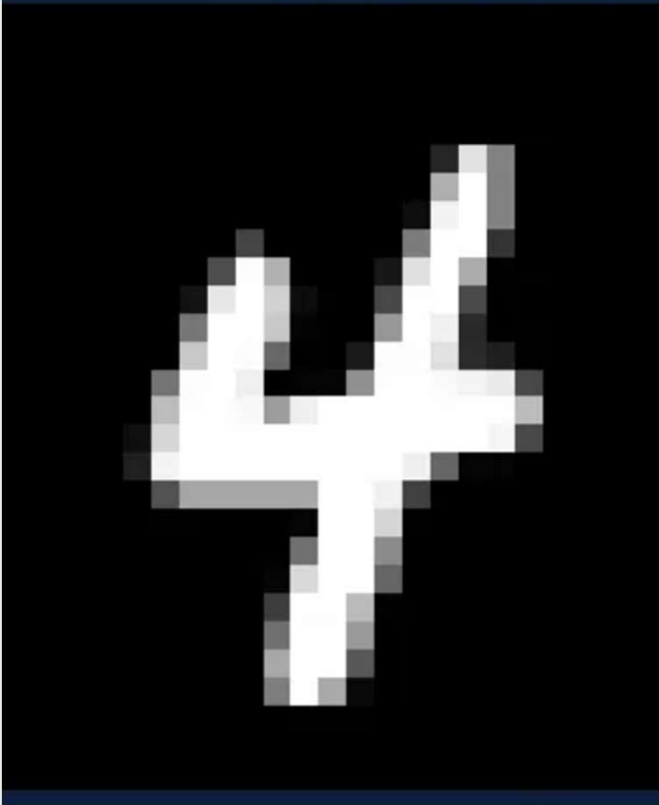
Output Size: (13, 13)
Receptive Field: 5*5

Size: 28*28

Index: 29385

Source: MNIST Source: /kaggle/input/mnist-dataset/train-images-idx3-ubyte/train-images-idx3-ubyte

Original Image



Kernel → ‘Sharpen effect’

Sharpen

Kernel Size (N*N)

3

5

[[0. -1. 0.]
[-1. 5. -1.]
[0. -1. 0.]]

PADDING

1

5

STRIDE

2

5

DILATION

2

5

Setup 1 Details

CONVOLUTION SETUP 1
DETAILS

Output Size: (13, 13)
Receptive Field: 5*5

Output 1

Kernel type: Sharpen

Setup 1 Details

CONVOLUTION SETUP 1 DETAILS

Output Size: (11, 11)
Receptive Field: 7×7

Output 1

Kernel type: Identity

Output 2

Kernel type: Identity

Setup 2 Details

CONVOLUTION SETUP 2 DETAILS

Output Size: (10, 10)
Receptive Field: 3×3

Size: 28×28

Index: 4619

Original Image

Kernel Type

Identity

Kernel Size (N×N)

☒ 3
 ☐ 5

Setup 1:

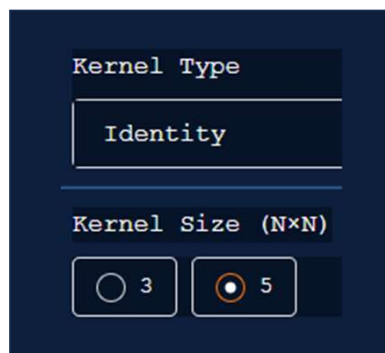
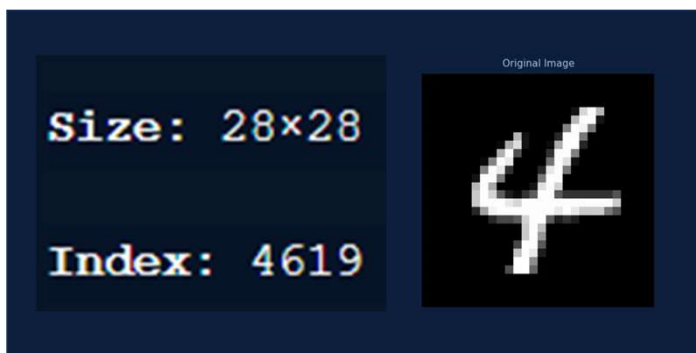
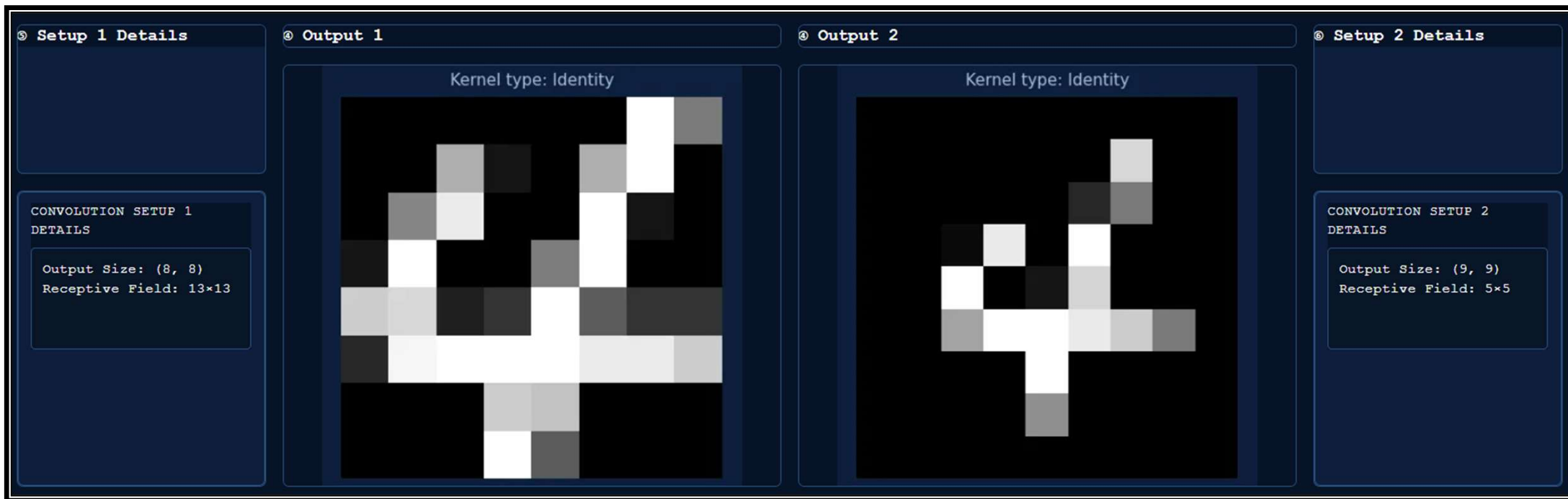
P = 0 / S = 2 / D = 3

Setup 2:

P = 1 / S = 3 / D = 1

Outcome:

similar output size but RF is completely different



Setup 1:	$P = 0 / S = 2 / D = 3$
Setup 2:	$P = 1 / S = 3 / D = 1$
Outcome:	improves RF at the cost of adding more parameters

Common Pitfalls

Improper Initialization of Dilated Modules:

Standard random initialization schemes are often ineffective for specialized modules using dilation (d). Use "identity initialization" to ensure that back-propagation can reliably harvest the contextual information provided by the dilated kernels.

Premature Resolution Loss from High Strides:

Using a high stride (s) or subsampling too early acts as a form of spatial subsampling that significantly reduces feature map resolution.

Successive Resolution Shrinking without Padding:

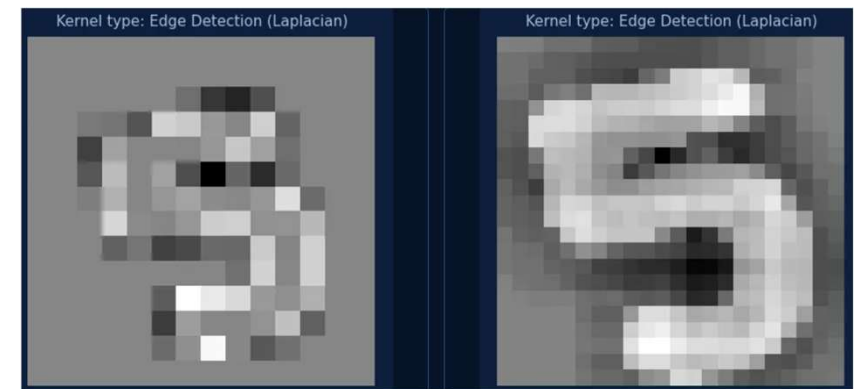
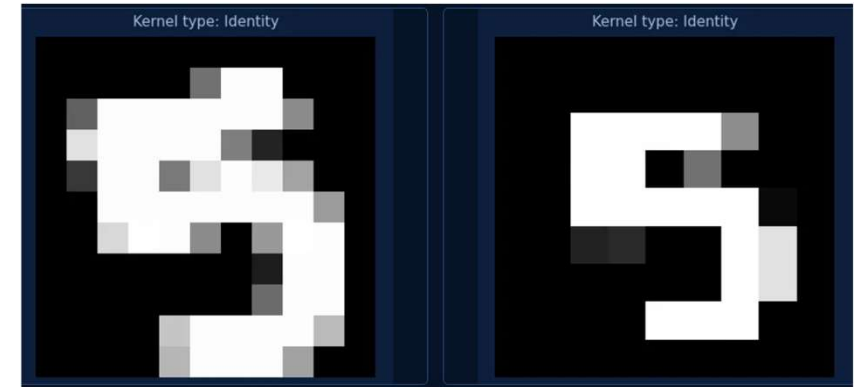
Failing to use adequate padding (p) causes the spatial dimensions of feature maps to shrink after each layer because kernels cannot be centered on edge pixels

Inefficient Receptive Field Expansion:

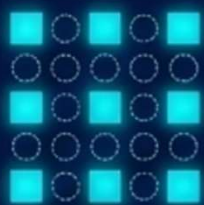
Relying solely on increasing kernel size (k) to expand the "field of view" is computationally expensive, as parameters grow quadratically

Overestimating Effective Context from Theoretical Fields:

A major conceptual pitfall is assuming that the theoretical receptive field—calculated using k , s , and d —represents the actual area used by the network.

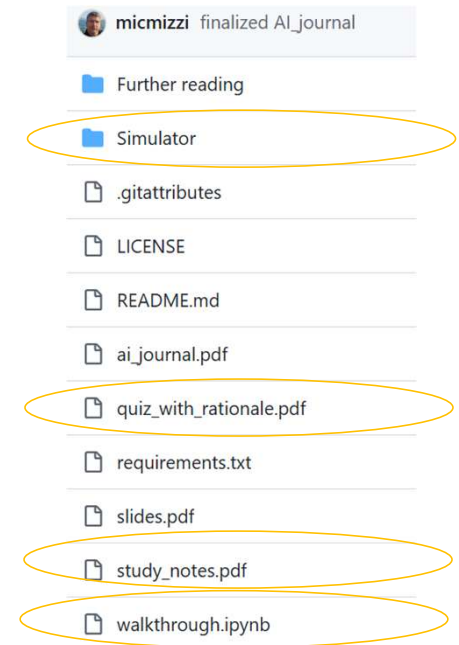


Summary

Visual Icon	Operation & Purpose	Effect on Output Size
	Standard Convolution: Local feature extraction.	Slight reduction ($i - k + 1$)
	Padded Convolution: Preserve spatial resolution.	Maintains exact size
	Strided Convolution: Spatial downsampling & compression.	Reduces size by a factor of s
	Dilated Convolution: Wide context aggregation.	Expands receptive field without losing resolution.

Next Steps.....

-Read through study notes
-follow the Jupyter notebook
-visualize the impact of *kernel size/type, padding, stride and dilation* using the simulator
-take the quiz
-further reading available as required



CNN Foundations ...the next building block ... **Pooling/Activation**